Automatic Differentiation

Machine Learning 1 Practicals

Thomas Wedenig

May 06, 2026

Institute of Machine Learning and Neural Computation
Graz University of Technology, Austria

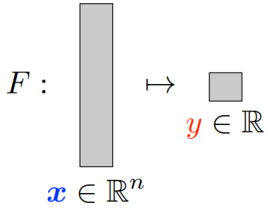
Automatic Differentiation

Credit: Matt Johnson's great talk on autodiff

[Johnson, 2017]

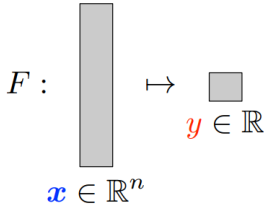
Every computational graph (DAG) defines a function that can be written as a composition of vector-valued functions!

$F : \mathbb{R}^n \to \mathbb{R}$



https://www.cs.toronto.edu/~duvenaud/talks/Johnson-Automatic-Differentiation.pdf

$$F : \mathbb{R}^n \to \mathbb{R}$$

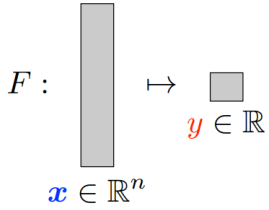


$$F = D \circ C \circ B \circ A$$

$$y = F(\boldsymbol{x}) = D(C(B(A(\boldsymbol{x}))))$$

https://www.cs.toronto.edu/~duvenaud/talks/Johnson-Automatic-Differentiation.pdf

$$F : \mathbb{R}^n \to \mathbb{R}$$



$$F = D \circ C \circ B \circ A \qquad y = F(\boldsymbol{x}) = D(C(B(A(\boldsymbol{x}))))$$

$$y = D(\boldsymbol{c}), \quad \boldsymbol{c} = C(\boldsymbol{b}), \quad \boldsymbol{b} = B(\boldsymbol{a}), \quad \boldsymbol{a} = A(\boldsymbol{x})$$

https://www.cs.toronto.edu/~duvenaud/talks/Johnson-Automatic-Differentiation.pdf

$$y = D(\boldsymbol{c}), \quad \boldsymbol{c} = C(\boldsymbol{b}), \quad \boldsymbol{b} = B(\boldsymbol{a}), \quad \boldsymbol{a} = A(\boldsymbol{x})$$

$$F'(\boldsymbol{x}) = \frac{\partial y}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} & \cdots & \frac{\partial y}{\partial x_n} \end{bmatrix}$$

https://www.cs.toronto.edu/~duvenaud/talks/Johnson-Automatic-Differentiation.pdf

$$y = D(\boldsymbol{c}), \quad \boldsymbol{c} = C(\boldsymbol{b}), \quad \boldsymbol{b} = B(\boldsymbol{a}), \quad \boldsymbol{a} = A(\boldsymbol{x})$$

$$F'(\boldsymbol{x}) = \frac{\partial y}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} & \cdots & \frac{\partial y}{\partial x_n} \end{bmatrix}$$

$$F'(\boldsymbol{x}) = \frac{\partial y}{\partial \boldsymbol{c}} \frac{\partial \boldsymbol{c}}{\partial \boldsymbol{b}} \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}} \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{x}}$$

https://www.cs.toronto.edu/~duvenaud/talks/Johnson-Automatic-Differentiation.pdf

$$y = D(\boldsymbol{c}), \quad \boldsymbol{c} = C(\boldsymbol{b}), \quad \boldsymbol{b} = B(\boldsymbol{a}), \quad \boldsymbol{a} = A(\boldsymbol{x})$$

$$F'(\boldsymbol{x}) = \frac{\partial y}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial y}{\partial x_1} & \cdots & \frac{\partial y}{\partial x_n} \end{bmatrix}$$

$$F'(\boldsymbol{x}) = \frac{\partial y}{\partial \boldsymbol{c}} \frac{\partial \boldsymbol{c}}{\partial \boldsymbol{b}} \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}} \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{x}}$$

$$\frac{\partial y}{\partial \boldsymbol{c}} = D'(\boldsymbol{c}) \qquad \frac{\partial \boldsymbol{c}}{\partial \boldsymbol{b}} = C'(\boldsymbol{b}) \qquad \frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}} = B'(\boldsymbol{a}) \qquad \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{x}} = A'(\boldsymbol{x})$$



https://www.cs.toronto.edu/~duvenaud/talks/Johnson-Automatic-Differentiation.pdf

$$F'(\boldsymbol{x}) = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{c}}\left(\frac{\partial \boldsymbol{c}}{\partial \boldsymbol{b}}\left(\underbrace{\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}}\quad\frac{\partial \boldsymbol{a}}{\partial \boldsymbol{x}}}\right)\right)$$

$$\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial b_1}{\partial x_1} & \cdots & \frac{\partial b_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial b_m}{\partial x_1} & \cdots & \frac{\partial b_m}{\partial x_n} \end{bmatrix}$$

Forward accumulation

$$F'(\boldsymbol{x}) = \frac{\partial y}{\partial \boldsymbol{c}}\left(\frac{\partial \boldsymbol{c}}{\partial \boldsymbol{b}}\underbrace{\left(\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}}\ \frac{\partial \boldsymbol{a}}{\partial \boldsymbol{x}}\right)}\right)$$

$$\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{x}} = \begin{bmatrix} \frac{\partial b_1}{\partial x_1} & \cdots & \frac{\partial b_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial b_m}{\partial x_1} & \cdots & \frac{\partial b_m}{\partial x_n} \end{bmatrix}$$

Forward accumulation

$$F'(\boldsymbol{x}) = \left(\underbrace{\left(\frac{\partial y}{\partial \boldsymbol{c}}\ \frac{\partial \boldsymbol{c}}{\partial \boldsymbol{b}}\right)}\frac{\partial \boldsymbol{b}}{\partial \boldsymbol{a}}\right)\frac{\partial \boldsymbol{a}}{\partial \boldsymbol{x}}$$

$$\frac{\partial y}{\partial \boldsymbol{b}} = \begin{bmatrix} \frac{\partial y}{\partial b_1} & \cdots & \frac{\partial y}{\partial b_m} \end{bmatrix}$$

Reverse accumulation

https://www.cs.toronto.edu/~duvenaud/talks/Johnson-Automatic-Differentiation.pdf

Forward vs. Reverse Autodiff (cont.)

Reverse-Mode Autodiff (“Backpropagation”)

- One backward pass just costs 1–3× **more compute than a forward pass** 😍
- But we need to store **all activations** in the **forward pass** 😓

Forward vs. Reverse Autodiff (cont.)

Reverse-Mode Autodiff ("Backpropagation")

- One backward pass just costs 1–3× **more compute than a forward pass** 😍
- But we need to store **all activations** in the **forward pass** 😓

Forward-Mode Autodiff

- Constant **memory overhead** 😍
- **Compute inefficient** if we compute gradients of $\mathcal{L} : \mathbb{R}^W \to \mathbb{R}$ 😩
- In **typical Deep Learning scenarios**, you rarely care about **forward-mode AD**

Feedback Time!

fbr.io/ml1fb1



References I



Johnson, M. J. (2017).
Automatic differentation.
Presented at the Deep Learning Summer School 2017 in Montreal.